Karnataka Residential Educational Institutions Society

ICT Class 10 - Practical Lab Exam 1

Practical Lab Exam 1 - Answer Key

Class: Class 10

Assessment Type: Practical Lab Exam 1

Total Marks: 20

Duration: 45 minutes

Topics Covered: HTML/CSS and Python Programming

Answer Key - Grading Guidelines

Note to Teacher: This answer key provides expected answers and marking guidelines. Accept equivalent answers where appropriate. Partial marks may be awarded for incomplete but correct responses.

Lab Tasks (15 marks)

Task 1: HTML/CSS - Personal Portfolio Website (5 marks)

Expected Solution:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Personal Portfolio</title>
  <style>
    * { margin: 0; padding: 0; box-sizing: border-box; }
    body { font-family: Arial, sans-serif; }
    .hero {
      background: linear-gradient(135deg, #667eea, #764ba2);
      color: white; text-align: center; padding: 100px 20px;
    }
    .hero h1 { font-size: 2.5em; }
    .skills { display: flex; flex-wrap: wrap; justify-content: center; padding:
40px; }
    .skill { width: 200px; margin: 10px; }
    .progress-bar { background: #ddd; border-radius: 5px; }
    .progress { background: #667eea; height: 20px; border-radius: 5px; }
    .projects { display: grid; grid-template-columns: repeat(auto-fit, minmax(250px,
1fr)); gap: 20px; padding: 40px; }
    .project-card { border: 1px solid #ddd; border-radius: 8px; padding: 15px; }
    form { max-width: 500px; margin: 40px auto; padding: 20px; }
    input, textarea { width: 100%; padding: 10px; margin: 5px 0; }
    @media (max-width: 768px) {
      .hero h1 { font-size: 1.8em; }
      .projects { grid-template-columns: 1fr; }
    }
  </style>
</head>
<body>
  <section class="hero">
    <h1>John Doe</h1>
    <p>Web Developer | Designer</p>
  </section>

  <section class="skills">
    <div class="skill"><h3>HTML</h3><div class="progress-bar"><div class="progress"
style="width:90%"></div></div></div>
    <div class="skill"><h3>CSS</h3><div class="progress-bar"><div class="progress"
style="width:85%"></div></div></div>
    <div class="skill"><h3>JavaScript</h3><div class="progress-bar"><div
```

```
class="progress" style="width:75%"></div></div></div>
    <div class="skill"><h3>Python</h3><div class="progress-bar"><div class="progress"
style="width:80%"></div></div></div>
    <div class="skill"><h3>SQL</h3><div class="progress-bar"><div class="progress"
style="width:70%"></div></div></div>
  </section>

  <section class="projects">
    <div class="project-card"><h3>Project 1</h3><p>Description</p></div>
    <div class="project-card"><h3>Project 2</h3><p>Description</p></div>
    <div class="project-card"><h3>Project 3</h3><p>Description</p></div>
  </section>

  <form>
    <input type="text" placeholder="Name">
    <input type="email" placeholder="Email">
    <textarea placeholder="Message"></textarea>
    <button type="submit">Send</button>
  </form>
</body>
</html>
```

Marking Criteria:

- Hero section with name and tagline: 1 mark
- Skills with progress bars: 1 mark
- Project cards (minimum 3): 1 mark
- Contact form: 1 mark
- Responsive design with media queries: 1 mark

Task 2: Python - Student Records Manager (5 marks)

Expected Solution:

```python
students = [
    {"name": "Rahul", "roll": 1, "marks": [85, 90, 78]},
    {"name": "Priya", "roll": 2, "marks": [92, 88, 95]},
    {"name": "Amit", "roll": 3, "marks": [78, 82, 75]}
]

def add_student():
    name = input("Enter name: ")
    roll = int(input("Enter roll number: "))
    marks = [int(input(f"Subject {i+1}: ")) for i in range(3)]
    students.append({"name": name, "roll": roll, "marks": marks})
    print("Student added!")

def display_all():
    print(f"{'Name':<15}{'Roll':<8}{'Total':<8}{'Avg':<8}")
    for s in students:
        total = sum(s["marks"])
        avg = total / 3
        print(f"{s['name']:<15}{s['roll']:<8}{total:<8}{avg:<8.2f}")

def search_student():
    roll = int(input("Enter roll number: "))
    for s in students:
        if s["roll"] == roll:
```

```
            print(f"Found: {s['name']}, Marks: {s['marks']}")
            return
    print("Student not found!")

def sorted_by_total():
    sorted_students = sorted(students, key=lambda x: sum(x["marks"]), reverse=True)
    display_all()

def statistics():
    totals = [sum(s["marks"]) for s in students]
    print(f"Average: {sum(totals)/len(totals):.2f}")
    print(f"Highest: {max(totals)}")
    print(f"Lowest: {min(totals)}")

while True:
    print("\n1.Add 2.Display 3.Search 4.Sorted 5.Stats 6.Exit")
    choice = input("Enter choice: ")
    if choice == "1": add_student()
    elif choice == "2": display_all()
    elif choice == "3": search_student()
    elif choice == "4": sorted_by_total()
    elif choice == "5": statistics()
    elif choice == "6": break
```

Marking Criteria:

- List of dictionaries structure: 1 mark
- Menu-driven interface: 1 mark
- Search and sort functions: 1 mark
- Statistics calculation: 1 mark
- Input validation and formatting: 1 mark

Task 3: Python - File Handling with Exception (5 marks)

Expected Solution:

```
import collections

def analyze_file(filename):
    try:
        with open(filename, 'r') as f:
            content = f.read()

        lines = content.split('\n')
        words = content.split()
        chars = len(content)
        unique_words = set(words)

        word_count = collections.Counter(words)
        top5 = word_count.most_common(5)

        with open('summary.txt', 'w') as f:
            f.write(f"File Analysis Report\n")
            f.write(f"{'='*30}\n")
            f.write(f"Total Lines: {len(lines)}\n")
            f.write(f"Total Words: {len(words)}\n")
            f.write(f"Total Characters: {chars}\n")
            f.write(f"Unique Words: {len(unique_words)}\n")
```

```
            f.write(f"\nTop 5 Words:\n")
            for word, count in top5:
                f.write(f"  {word}: {count}\n")

        print("Summary written to summary.txt")

    except FileNotFoundError:
        print(f"Error: File '{filename}' not found!")
    except PermissionError:
        print(f"Error: No permission to read '{filename}'!")
    except Exception as e:
        print(f"Error: {str(e)}")

filename = input("Enter filename: ")
analyze_file(filename)
```

Marking Criteria:

- File reading with with statement: 1 mark
- Counting logic (words, lines, characters): 1 mark
- Unique words and frequency: 1 mark
- Exception handling (3 types): 1 mark
- Report writing to file: 1 mark

Viva Voce (5 marks)

1. **display: none vs visibility: hidden**
   - **Answer:** `display: none` removes element from document flow (no space reserved). `visibility: hidden` hides element but space is preserved. Use `display: none` when element should not affect layout; use `visibility: hidden` when you want to toggle visibility without layout shift.
   - **Marking:** 1 mark for correct explanation.
2. **CSS Box Model**
   - **Answer:** Box model: content → padding → border → margin. `box-sizing: border-box` includes padding and border in the element's total width/height, making layout calculations easier.
   - **Marking:** 1 mark for box model explanation.
3. **with open() preference**
   - **Answer:** `with open()` automatically closes the file after the block, even if exceptions occur. Manual open()/close() requires explicit close and may leak resources if exceptions happen before close().
   - **Marking:** 1 mark for explanation.
4. **read(), readline(), readlines()**
   - **Answer:** `read()` returns entire file as string. `readline()` returns one line. `readlines()` returns list of all lines. Use `read()` for small files, `readline()` for line-by-line, `readlines()` when list needed.
   - **Marking:** 1 mark for differences.
5. **Python exception handling**
   - **Answer:** Python uses `try-except-else-finally`. `try` contains risky code, `except` handles errors, `else` runs if no exception, `finally` always runs (cleanup). Purpose: gracefully handle errors without crashing.

- **Marking:** 1 mark for explanation.